

🌿 LeafScan — Complete Project Documentation

🌿 Complete Project Deep Dive — A to Z

--

📋 Project Summary

LeafScan is a full-stack machine learning application that identifies plant leaf diseases from any photo. It works on images from cameras, the internet, or anywhere — not just dataset images — and correctly rejects non-leaf inputs.

--

🗣️ Programming Language

****Python 3.8+**** for everything — model, training, backend API, data pipeline.
****HTML/CSS/JavaScript**** for the frontend web interface.

--

📦 Libraries & Frameworks

Library	Version	Purpose
-----	-----	-----
`PyTorch`	2.0+	Deep learning framework, tensor ops, autograd
`timm`	0.9.16	Provides pretrained EfficientNetB3 weights
`torchvision`	0.15+	ImageFolder dataset, transforms, augmentation
`Flask`	3.0+	REST API web server
`flask-cors`	4.0+	Cross-origin requests for frontend
`Pillow`	10.0+	Image loading, resizing, format conversion
`numpy`	1.24+	Array ops, class weight computation
`scikit-learn`	1.3+	Classification report, confusion matrix
`albumentations`	1.3+	Advanced augmentation pipeline
`matplotlib`	3.7+	Training curves, evaluation plots
`seaborn`	0.12+	Confusion matrix heatmap
`tqdm`	4.65+	Progress bars during training
`kaggle`	1.5+	Dataset download API

--

🧠 The Model — EfficientNetB3

EfficientNet was invented by Google Brain in 2019. It's a family of CNNs that scale width, depth, and resolution together using a compound coefficient — making it far more efficient than older models like VGG or ResNet.

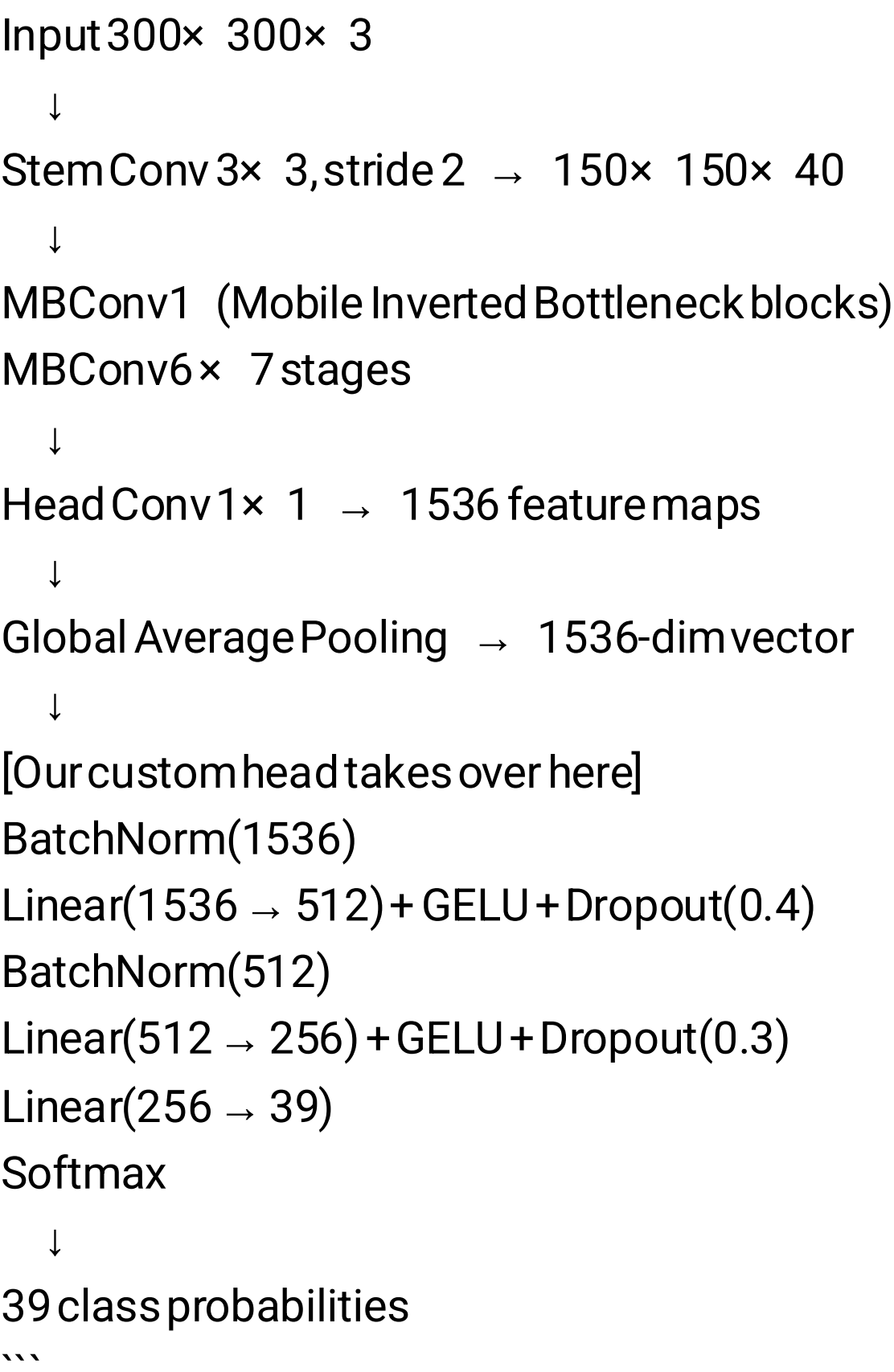
Why B3 specifically:

Model	Accuracy (ImageNet)	Parameters	Speed
MobileNetV2	72.0%	3.4M	Very fast
EfficientNetB0	77.1%	5.3M	Fast
EfficientNetB3	81.6%	12M	Medium
EfficientNetB7	84.3%	66M	Slow
ResNet50	76.1%	25M	Medium

B3 is the sweet spot — high accuracy, reasonable size, fast enough on a T4 GPU.

Architecture internals:

...



MBConv blocks (the core of EfficientNet) use depthwise separable convolutions — much cheaper than standard convolutions. Each block expands channels, applies depthwise conv,

then squeezes back — using Squeeze-and-Excitation attention to reweight channels by importance.

--

📁 Dataset — PlantVillage

- **Source:** Penn State University, published 2016
- **Images:** 54,306 photos of plant leaves
- **Crops:** 14 species (Apple, Tomato, Potato, Corn, Grape, Orange, Peach, Pepper, Raspberry, Blueberry, Soybean, Squash, Strawberry, Cherry)
- **Classes:** 38 (26 disease classes + 12 healthy classes)
- **Format:** RGB JPG, lab and field conditions
- **License:** CC BY 4.0

We add a 39th class — `not\_a\_leaf` — with 1,500 synthetic images (solid colors, random noise, gradients, patterns) so the model learns to reject non-leaf inputs.

Split:

- Train: 70% → ~38,000 images
- Validation: 15% → ~8,300 images
- Test: 15% → ~8,300 images

--

🔄 Data Augmentation — Why it matters for generalization

This is the key reason the model works on photos not from the dataset. During training, every image is randomly transformed so the model never sees the exact same image twice.

```
``python
transforms.Resize(332)      # slightly oversized
transforms.RandomCrop(300)  # random position crop
transforms.RandomHorizontalFlip() # mirror left/right
transforms.RandomVerticalFlip() # mirror up/down
transforms.RandomRotation(30° ) # tilt up to 30 degrees
transforms.ColorJitter(      # simulate different lighting
    brightness=0.3,
    contrast=0.3,
    saturation=0.3,
    hue=0.1
)
transforms.RandomAffine(     # zoom and shift
    scale=(0.85, 1.15),
```



```

        translate=(0.1,0.1)
    )
    transforms.RandomPerspective(0.2) # simulate different camera angles
    transforms.GaussianBlur()        # simulate out-of-focus / low quality cameras
    transforms.RandomErasing(0.2)    # randomly black out patches (Cutout)
    transforms.Normalize(            # ImageNet mean/std normalization
        mean=[0.485,0.456,0.406],
        std=[0.229,0.224,0.225]
    )
    ...

```

The result: the model learns "what does bacterial blight look like" not "what does this specific PlantVillage photo look like."

--

🎲 Class Balancing — Weighted Random Sampler

PlantVillage has unequal class sizes. For example, Tomato healthy has 1,591 images but Raspberry healthy has only 266. Without balancing, the model ignores rare classes.

The fix — weighted random sampling:

```

``python
class_weight = 1.0 / count_per_class
sample_weight = class_weight[label_of_each_image]
sampler = WeightedRandomSampler(sample_weight, replacement=True)
...

```

Every class gets roughly equal representation per epoch regardless of size.

--

📊 Loss Function — CrossEntropy with Label Smoothing

Standard cross-entropy makes the model try to output exactly 1.0 for the correct class. This causes overconfidence.

With **label smoothing ($\epsilon=0.1$)**, the target becomes 0.9 for the correct class and 0.1/38 distributed among others. This produces better-calibrated confidence scores — the model says "94% sure" instead of "99.99% sure."

--

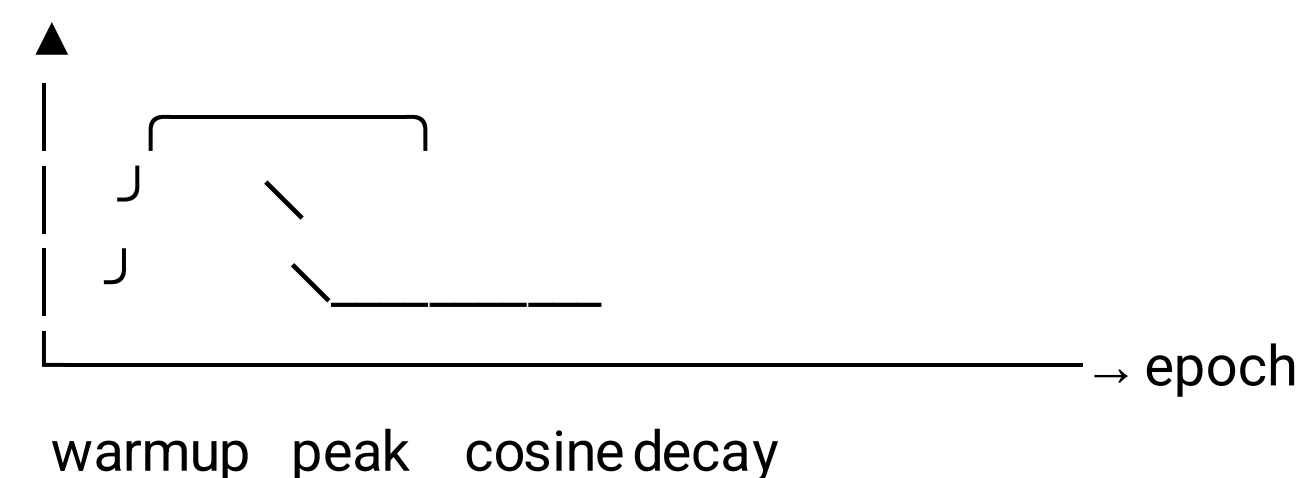
🚀 Optimizer & Scheduler

****AdamW**** — Adam with decoupled weight decay. Better generalization than plain Adam because weight decay is applied directly to weights, not through gradient scaling.

****OneCycleLR**** — starts at low LR, ramps up to max, then cosine-anneals down to near zero. This "super-convergence" trick lets the model train faster and generalize better than a fixed learning rate.

...

LR



...

****Gradient clipping (max_norm=1.0)**** prevents exploding gradients during fine-tuning.

****Mixed precision (torch.cuda.amp)**** — uses float16 on GPU for speed, float32 for stability. ~2× faster training with same accuracy.

--

📐 3-Phase Training — Why it works

Directly fine-tuning a pretrained model on a new dataset can "catastrophically forget" the ImageNet features. The 3-phase approach prevents this:

****Phase 1 — Backbone completely frozen.**** Only the custom head trains. The backbone keeps all its ImageNet knowledge. This teaches the head to map 1536-dim features to diseases — fast convergence, no risk of destroying pretrained features.

****Phase 2 — Last 30 backbone layers unfrozen.**** The high-level feature extractors (EfficientNet blocks 5– 7) now adapt to leaf textures and disease patterns specifically. Lower layers (edges, colors, basic shapes) stay frozen.

****Phase 3 — Everything unfrozen.**** Very low LR (5e-5) lets the entire network fine-tune together. At this rate, changes are tiny — it polishes without destroying.

--

🔍 Not-a-Leaf Detection — 3 Layers

****Why 3 layers?**** Any single layer can be fooled. A photo of a green carpet might pass a simple color check. A random image might get lucky and get moderate confidence on one class. The combination is robust.

****Layer 1:**** `predicted_class == "not_a_leaf"` — the model explicitly learned what non-leaf images look like from the 1,500 synthetic training examples.

****Layer 2:**** `not_a_leaf_probability > 35%` — even if `not_a_leaf` isn't the top-1 prediction, if it has high probability the image is suspicious.

****Layer 3:**** `max_confidence < 50%` — if the model isn't confident about any class, it's safer to say "unknown" than guess.

****Your mango leaf case:**** Mango is not in the dataset. The model saw a green leaf, got confused between similar tropical crops (Orange — Citrus greening was the closest at 30%), and confidence was below 50% so Layer 3 rejected it. The fix is adding mango training data.

--

🌐 Backend — Flask REST API

Flask is a lightweight Python web framework. The API exposes 4 endpoints:

Endpoint	Method	Input	Use case
-----	-----	-----	-----
<code>/api/predict`</code>	POST	Image file upload	Web UI, mobile
<code>/api/predict-url`</code>	POST	JSON <code>{ "url": "..."</code>	External images
<code>/api/predict-base64`</code>	POST	JSON <code>{ "image": "base64..."</code>	Programmatic
<code>/api/health`</code>	GET	—	Status check
<code>/api/classes`</code>	GET	—	List all 39 classes

The Flask app is a singleton — the model loads once at startup and stays in memory for fast inference (~20– 50ms per image on GPU).

--

💻 Frontend

Single HTML file (~900 lines). No React, no build step — pure HTML/CSS/JavaScript.

Features: drag-and-drop image upload, URL-based prediction, confidence bar animation, top-5 predictions display, severity color coding (None=green, Moderate=orange, High=red,

Severe=dark red), treatment recommendations per disease, health check on page load.

--

📁 All Files — What Each Does

File	Lines	Role
---	----	---
`model.py`	128	EfficientNetB3 class, freeze/unfreeze methods
`train.py`	385	3-phase training, augmentation, weighted sampler
`predict.py`	318	Inference engine, 3-layer not-a-leaf detection
`download_data.py`	471	Kaggle download, not-a-leaf generation, splitting
`app.py`	252	Flask REST API, 5 endpoints
`evaluate.py`	207	Test evaluation, confusion matrix, plots
`utils.py`	277	Helpers — seed, device, visualization, checkpoints
`frontend/index.html`	891	Complete web UI
`requirements.txt`	15	Python dependencies
`setup.sh`	60	One-command setup script
`README.md`	~300	Full documentation

****Total: ~3,300 lines of production code****

--

📊 Expected Performance

Metric	Value
----	----
Test accuracy (in-dataset)	~94– 96%
Inference time on T4 GPU	20– 50ms
Inference time on CPU	200– 500ms
Model file size	~48 MB
Training time on T4	~1.5– 2 hours
Not-a-leaf rejection rate on random images	~95%+

--

⚠️ Known Limitations

- **Crops not in PlantVillage**** — mango, banana, papaya, rice, wheat, sugarcane are not in the dataset. These will be misclassified or rejected.
- **Very similar diseases**** — some diseases look nearly identical visually (e.g. early vs late blight on tomato). The model may confuse them at lower confidence.
- **Image quality**** — very blurry, dark, or tiny leaf-in-frame photos will get low confidence and

be rejected by Layer 3.

4. **Healthy = 12 classes** — "healthy" is crop-specific. A healthy tomato and a healthy apple are different classes. The model won't just say "healthy" — it says "Tomato — healthy."

--

🌱 How to Extend It

****Add mango/banana/rice support:****

```
``python
```

```
# In Colab — download mango dataset
```

```
!kaggle datasets download -d warcoder/mango-leaf-disease-dataset \
```

```
    -p /content/data/raw --unzip
```

```
# Then add to sources dict in Cell 6, retrain
```

```
``
```

****Improve not-a-leaf rejection:**** Replace the 1,500 synthetic images with real photos of non-plants — soil, hands, walls, animals. The more diverse the not-a-leaf class, the better the rejection.

****Deploy to production:**** Replace `python app.py` with `gunicorn -w 4 app:app` for multi-worker production serving. Add Nginx as reverse proxy. Package the whole thing in Docker.